Design Approach:

As per the project requirements, the key goal of the device is to provide protection against over-voltage and over-current. I chose a simple approach: there will be sensors for voltage and current which will provide feedback to the MCU; an electromechanical relay will be connected to the main line between the source and the load. If the set threshold is crossed, the MCU will command the relay to cut the line. I have used an Arduino Uno as the microcontroller, an LU-5-R module as the relay, a voltage divider circuit as the voltage sensor, and two potentiometers for threshold voltage and current settings. A DC bulb and a DC motor are connected in parallel as the load, along with some LEDs, a buzzer, and an LCD to alert and monitor the status of the circuit.

Working Principle:

I have used a voltage source (0–30 VDC) as the power supply. A voltage divider circuit is directly connected to it. This voltage divider has two resistors: a 47-ohm resistor connected to the positive side of the source, and a 220-ohm resistor connected to the other side of the 47-ohm resistor and to the ground. A line is taken from the junction of the two resistors, which is then mapped from raw analog values (0–1023) to 0–5V. The resistors are chosen such that the analog input to the MCU cannot exceed 5V, which may damage the Arduino pins; this is how the voltage is sensed. The main line is connected through a relay module in a “normally closed” position, so the load is always connected to the power if no issues appear in the circuit. If the measured voltage crosses the threshold, the MCU will immediately send a signal to the relay to open the circuit. The threshold voltage and current values can be calibrated by physical knobs on the device, and the user can change the values in real time. An LCD display shows the threshold voltage and current set by the user, as well as the current state of the circuit (e.g., ‘NORMAL’, ‘OVERVOLTAGE’, ‘OVERCURRENT’). Moreover, during a faulty condition, a red LED will blink and the buzzer will buzz. During a normal condition, a green LED blinks. So, the system can detect a overvoltage situation and can take appropriate action to mitigate the issue.

Implementation:

I have simulated the entire system on Tinkercad.

Limitations:

Due to the lack of library support in the simulation software, I did not manage to add any current sensor to sense the current. However, my code is well capable of detecting and taking action during an overcurrent condition. If current is fed to the MCU, the program can work with such a condition. In real life, the current can simply be measured by an ACS712 current sensor module.